

MODULE 1 — INTRODUCTION TO FILE HANDLING IN JAVA

Learning Objectives

By the end of this module, students should be able to:

1. Understand what file handling is and why it is needed.
2. Identify and explain key Java classes used for file operations.
3. Create, inspect, and manage files and directories using the `File` class.
4. Handle basic file creation and deletion operations.

1.1 Introduction to File Handling

◆ Definition

File handling in Java is the process of **creating**, **reading**, **writing**, and **managing** files and directories using Java's built-in APIs. It allows programs to store and retrieve data permanently from the storage system (disk).

Why Do We Need File Handling?

When a program runs, data stored in variables and memory is **temporary** — it disappears when the program stops.

File handling provides a way to **store data permanently**.

Examples of where file handling is used:

- Saving user information in text files.
- Logging activities (e.g., system logs, error logs).
- Reading and writing configuration settings.
- Managing documents, reports, and data files.



1.2 File Handling Packages in Java

Java provides two main packages for handling files:

Package	Description	Key Classes
java.io	Traditional Input/Output package for file handling (stream-based).	File , FileWriter , FileReader , BufferedReader , etc.
java.nio.file	New I/O (NIO) package introduced in Java 7 for advanced file operations.	Files , Path , Paths , etc.

In this module, we focus on the **File class** from the `java.io` package.



1.3 The File Class



Overview

- The `File` class is found in the `java.io` package.
- It represents a file or directory in the file system (it does **not** store data itself — it is a path abstraction).
- With it, we can:
 - Create a new file or directory.
 - Check if a file exists.
 - Retrieve information like name, size, and permissions.
 - Delete files or directories.



Importing the File Class

```
import java.io.File;
```

◆ Creating a File Object

```
File file = new File("example.txt");
```

This creates a **File object**, not an actual file on disk yet.

To actually create the file, you must call `createNewFile()` .



Common Constructors

Constructor	Description
<code>File(String pathname)</code>	Creates a File object for the given path
<code>File(String parent, String child)</code>	Creates a File using parent and child directory
<code>File(File parent, String child)</code>	Creates a File from an existing File directory and child file name

Example:

```
File f1 = new File("data.txt");  
File f2 = new File("C:/Users/Lee/Documents", "notes.txt");  
File folder = new File("C:/Users/Lee/Projects");
```



1.4 Useful Methods of the File Class

Method	Description
<code>createNewFile()</code>	Creates a new file if it doesn't exist
<code>mkdir()</code>	Creates a new directory
<code>mkdirs()</code>	Creates directories including any necessary parent directories
<code>exists()</code>	Checks if the file or folder exists
<code>delete()</code>	Deletes the file or directory
<code>getName()</code>	Returns the name of the file or directory
<code>getAbsolutePath()</code>	Returns the full path of the file

Method	Description
length()	Returns the size (in bytes)
canRead()	Checks if the file is readable
canWrite()	Checks if the file is writable
isFile()	Checks if it's a file
isDirectory()	Checks if it's a directory
list()	Lists the contents of a directory



1.5 Practical Examples



Example 1: Creating a New File

```
import java.io.File;
import java.io.IOException;

public class CreateFileExample {
    public static void main(String[] args) {
        try {
            File file = new File("example.txt");
            if (file.createNewFile()) {
                System.out.println("File created: " + file.getName());
            } else {
                System.out.println("File already exists.");
            }
        } catch (IOException e) {
            System.out.println("An error occurred while creating the file.");
            e.printStackTrace();
        }
    }
}
```

Explanation:

- `createNewFile()` actually creates the file if it doesn't exist.
- It throws an `IOException` if something goes wrong.

✓ Example 2: Checking File Information

```
import java.io.File;

public class FileInfoExample {
    public static void main(String[] args) {
        File file = new File("example.txt");

        if (file.exists()) {
            System.out.println("File name: " + file.getName());
            System.out.println("Absolute path: " + file.getAbsolutePath());
            System.out.println("Writable: " + file.canWrite());
            System.out.println("Readable: " + file.canRead());
            System.out.println("File size: " + file.length() + " bytes");
        } else {
            System.out.println("The file does not exist.");
        }
    }
}
```

✓ Example 3: Creating a Directory

```
import java.io.File;

public class CreateDirectoryExample {
    public static void main(String[] args) {
        File dir = new File("MyFolder");
        if (dir.mkdir()) {
            System.out.println("Directory created: " + dir.getName());
        } else {
            System.out.println("Directory already exists or could not be created.");
        }
    }
}
```

Use `makedirs()` if you want to create nested folders (e.g., "Parent/Child/Subfolder").

✓ Example 4: Listing Files in a Directory

```
import java.io.File;

public class ListFilesExample {
    public static void main(String[] args) {
        File folder = new File("C:/Users/Lee/Documents");
        String[] files = folder.list();

        if (files != null) {
            for (String name : files) {
                System.out.println(name);
            }
        } else {
            System.out.println("Folder is empty or does not exist.");
        }
    }
}
```

✓ Example 5: Deleting a File

```
import java.io.File;

public class DeleteFileExample {
    public static void main(String[] args) {
        File file = new File("example.txt");
        if (file.delete()) {
            System.out.println("Deleted the file: " + file.getName());
        } else {
            System.out.println("Failed to delete the file.");
        }
    }
}
```

⚠ 1.6 Exception Handling in File Creation

Many file operations throw an `IOException`, which must be handled with a `try-catch` block.

Example:

```
try {  
    File file = new File("data.txt");  
    file.createNewFile();  
} catch (IOException e) {  
    System.out.println("Error creating file: " + e.getMessage());  
}
```



1.7 Summary of Module 1

Concept	Description
File Handling	Process of creating, reading, writing, and managing files
File Class	Represents files and directories
createNewFile()	Creates a new file
mkdir() / mkdirs()	Creates directories
exists()	Checks if file/directory exists
delete()	Deletes file or folder
getName() / getAbsolutePath()	Retrieves file details



Classwork / Practical Tasks

1. Write a Java program to create a file named **"students.txt"** and print a message if the file is newly created or already exists.
2. Write a Java program that prints the **name**, **path**, and **size** of the file "students.txt".
3. Create a folder named **"Assignments"** and print its absolute path.
4. Create another folder inside "Assignments" called "Java" using `mkdirs()` .
5. Write a program that lists all files in your working directory.
6. Write a program that deletes the "students.txt" file.



Discussion Question

What is the difference between creating a `File` object and actually creating a physical file on disk?

(Hint: Think about the `new File()` constructor vs `createNewFile()` method.)

🏁 End of Module 1 Summary

You've now learned how to:

- Create file objects.
- Inspect file information.
- Manage directories.
- Delete and list files.
- Handle basic I/O exceptions.

In the next module, we'll dive into **reading and writing data into files** using `FileReader` , `FileWriter` , `BufferedReader` , and `BufferedWriter` .